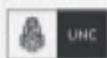


CERTIFICACIÓN UNIVERSITARIA



Manual para el
alumno

Docker II



**WE WANT
SKILLS!**

mundosE

Manual para el Alumno: Introducción a Docker II.....	2
Objetivo del encuentro:.....	2
Contenido a desarrollar:.....	2
1. Imágenes Docker: concepto y ciclo de vida.....	3
2. Dockerfile paso a paso: construcción de imágenes propias.....	4
3. Volúmenes: persistencia de datos.....	5
4. Redes Docker: bridge, host y redes personalizadas.....	6
5. Docker Compose y registries privados.....	7
Bibliografía.....	9

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontraremos un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el Alumno: Introducción a Docker II

Objetivo del encuentro:

Consolidar aprendizajes sobre Docker al construir imágenes personalizadas con Dockerfile, gestionar datos persistentes mediante volúmenes, configurar redes entre contenedores y orquestar pilas de servicios con Docker Compose.

Al finalizar el módulo podrán:

- Describir el ciclo de vida completo de una imagen Docker y su arquitectura en capas.
- Redactar Dockerfiles óptimos, aplicando *multi-stage*, caché eficaz y usuario *no-root*.
- Determinar el tipo de volumen adecuado para cada aplicación y plantear planes de *backup*.
- Evaluar los drivers de red disponibles y elegir el apropiado según rendimiento y seguridad.
- Definir una pila multiservicio con Compose y explicar la importancia de los *registries* privados, la firma y el escaneo de imágenes.

Contenido a desarrollar:

- ✓ Imágenes Docker: concepto y ciclo de vida
- ✓ Dockerfile paso a paso: construcción de imágenes propias
- ✓ Volúmenes: persistencia de datos
- ✓ Redes Docker: *bridge*, *host* y redes personalizadas
- ✓ Docker Compose y registries privados

1. Imágenes Docker: concepto y ciclo de vida

La imagen Docker es la unidad atómica de despliegue: un paquete inmutable que contiene sistema de archivos, binarios, librerías y metadatos. Su magia reside en la **arquitectura en capas**. Cada instrucción del Dockerfile genera una layer de solo lectura identificada por un SHA-256 que la vuelve inmutable y compartible. Cuando ejecutamos `docker run`, el demonio monta las capas en cadena y agrega una capa RW *copy-on-write* encima; de ese modo, los cambios que el proceso realice en runtime no afectan a la imagen de base y se descartan al borrar el contenedor.

Esta disposición habilita tres superpoderes de la vida moderna DevOps.

Primero, la **caché de build**: si las primeras instrucciones (`FROM`, instalación de dependencias) no cambiaron, Docker reutiliza esas capas y solo rehace las posteriores, acelerando la compilación. Segundo, la **distribución diferencial**: `docker pull` consulta al registry qué capas componen una imagen y descarga únicamente las ausentes. Por último, la **inmutabilidad verificable**: al referirnos al digest (`sha256:...`) garantizamos que QA y producción ejecuten exactamente los mismos bits.

Desde la perspectiva de trazabilidad, la Open Container Initiative (OCI) define el manifiesto que lista capas, configuración y *digest*; cualquier *runtime* compatible (containerd, CRI-O, Podman) puede ejecutar esa imagen. En auditorías, el *digest* confiere no repudio: si la imagen tiene firma criptográfica, el sistema puede verificar que nadie la alteró después del pipeline.

La práctica recomienda **versionar doblemente**: etiqueta semántica (`1.4.3`) para lectura humana y *digest* para la máquina. Asimismo, conviene minimizar capas fusionando comandos relacionados; un solo `RUN apt-get update && apt-get install -y ... && rm -rf /var/lib/apt/lists/*` evita repetir tres *layers* innecesarias y reduce tamaño. A su vez, las **multi-stage builds** permiten compilar en un *stage* voluminoso y copiar el artefacto resultante a un *stage* mínimo (Alpine, distroless).

En entornos corporativos, la imagen fluye por cuatro fases: *build* (creación), *push* (publicación), *pull* (descarga) y *run* (ejecución). Este pipeline reemplaza el modelo tradicional de “configurar servidores”; ahora la infraestructura se versiona como código, y la imagen es la verdad única que viaja entre ambientes.

Un punto crítico es el **escaneo de vulnerabilidades**. Herramientas como Trivy o Snyk analizan cada capa contra bases CVE; si detectan un defecto crítico no parchado, bloquean el *push*. Así se integra Seguridad desde la etapa de construcción, no como validación tardía.

Por último, es clave entender que el tamaño no lo es todo: una imagen muy pequeña basada en Alpine puede carecer de *glibc*, dificultando el debug. El criterio debe equilibrar *footprint*, funcionalidad y velocidad de arranque.

Recursos para profundizar:

- **Docker** © 2013-2025. Digests. <https://docs.docker.com/dhi/core-concepts/digests/>
- **Docker** © 2013-2025. Docker Docs – Images, layers, and caching: <https://docs.docker.com/build/cache/>
- **CitHub** © (2025). Open Container Initiative – OCI Image Spec v1.0.2: <https://github.com/opencontainers/image-spec>
- **IBM** © (2025). Digests versus image tags. <https://www.ibm.com/docs/en/filenet-p8-platform/5.6.0?topic=deployment-digests-versus-image-tags>

2. Dockerfile paso a paso: construcción de imágenes propias

El Dockerfile es la **receta reproducible** que describe cómo se arma cada capa. Su ejecución es lineal: si modificamos la línea 4, todo lo que esté por debajo perderá la caché. Por eso la regla de oro es colocar arriba lo que casi nunca cambia (instalación de *toolchains*) y al final lo frecuente (código fuente).

Una plantilla típica empieza con **FROM** y una imagen base oficial. A continuación se agregan **LABEL** de autor y commit para trazabilidad. Luego vienen las dependencias; por ejemplo, en Node se copian los **package-lock.json**, se ejecuta **npm ci --only=prod** y recién después se copia el código. Así, cambiar un archivo **.js** no invalida la costosa instalación de módulos.

Los comandos **RUN** deberían agruparse y limpiar el caché del gestor (**apt**, **apk**) dentro de la misma línea. Eso evita capas sobrantes y minimiza tamaño. Para configuraciones variables, **ARG** permite pasar valores en *build time* sin “ensuciar” la capa final, mientras que **ENV** fija variables persistentes en *runtime*.

Multi-stage build: imaginemos una API Go. El stage **builder** compila en **golang:1.22-alpine**, y el stage final copia el binario a **gcr.io/distroless/base-debian12**, reduce superficie de ataque, elimina *toolchain* y entrega una imagen de 20 MB. Esta estrategia habilita *static linking*, menor *CVE count* y despliegues más veloces.

La seguridad inicia con **USER app (no-root)** y la caída de capacidades que no se necesitan (**CAP_SYS_ADMIN**, **CAP_NET_RAW**). Incluir una política **HEALTHCHECK** permite a orquestadores detectar contenedores *zombies*.

Herramientas como **hadolint** analizan el Dockerfile y reportan malas prácticas:

uso de **latest**, cachés no limpiados, falta de *digest* en imagen base. **BuildKit** introduce soporte para caché remoto, secretos sin persistir (ideal para claves npm) y ejecuciones paralelas; basta exportar **DOCKER_BUILDKIT=1**. **Dive** muestra cuánto pesa cada *layer* y qué archivos aporta, ayudando a cortar grasa innecesaria.

Finalmente, mantener un **.dockerignore** evita copiar a la *build context* carpetas pesadas (**.git**, **node_modules**) o secretos locales. El tamaño del contexto incide en tiempo de envío al daemon, sobre todo desde CI remotos.

Recursos para profundizar:

- **Docker** © 2013-2025. BuildKit official docs – caching & secrets: <https://docs.docker.com/build/buildkit/>
- **GitHub** © (2025). hadolint – linter de Dockerfiles. <https://github.com/hadolint/hadolint>
- **GitHub** © (2025). Repositorio Dive – análisis de capas. <https://github.com/wagoodman/dive>
- **Karlsson, D.** (2023, nov 24). [Docker] Dockerfile: From Start to Optimized (DockerCon 2023). <https://www.youtube.com/watch?v=saSJa9YVroA>

3. Volúmenes: persistencia de datos

Los contenedores han sido diseñados para ser descartables. Pero la mayoría de aplicaciones reales necesitan guardar estado: bases de datos, ficheros de usuario, *logs* regulados. Docker resuelve este dilema mediante **volúmenes**, abstracciones que se montan en el contenedor, pero viven fuera de su capa *copy-on-write*. Al borrar el contenedor, el volumen persiste.

Existen tres sabores. El **volumen anónimo** se crea al vuelo cuando especificamos **-v /datos**; Docker le asigna un ID críptico y es práctico para laboratorios donde no importa la ruta física. El **volumen nombrado** (**-v nombre:/datos**) permite reutilizarlo entre contenedores, copiarlo, o gestionar su ciclo de vida con **docker volume**. Por último, el **bind mount** conecta un directorio del host (**-v /home/user/html:/usr/share/nginx/html**): ideal en desarrollo para *hot-reload*, aunque acopla la ruta y hereda permisos.

A nivel rendimiento, los volúmenes con driver **local** evitan el *overlay*, escribiendo directo en ext4/xfs; eso reduce la latencia en bases OLTP. En Kubernetes, la abstracción pasa a ser el objeto **PersistentVolumeClaim** y se apoya en CSI *drivers* (AWS EBS, Ceph RBD).

Copias de seguridad:

- **Snapshots** (LVM, ZFS, EBS): congelan estado casi al instante, pero dependen del almacenamiento subyacente.

- **Dump lógico** (`mysqldump`, `pg_dump`): produce archivo transportable, independiente del motor.
- **Sync block** (`rsync`, `rsync S3`): combina rapidez con compresión y cifrado.

Una estrategia híbrida usa *snapshots* horarios para RPO bajo y *dumps* diarios para portabilidad. La restauración requiere recrear un volumen y rellenarlo; la *app* arranca con su estado intacto.

Seguridad: cifrado en tránsito (TLS) si el volumen viaja por red (NFS, iSCSI) y cifrado en reposo si la normativa lo exige. También es buena práctica mapear UID/GID correctos y montar la raíz del contenedor como `--read-only`, dejando solo volúmenes selectivos en modo RW.

Recursos para profundizar:

- **Docker** © 2013-2025. Docker Docs “Manage data in Docker”.
<https://docs.docker.com/storage/>
- **GitHub** © (2025). CSI spec – Container Storage Interface.
<https://github.com/container-storage-interface/spec>
- **Percona** © (2025). Run in a Docker container. Percona XtraBackup 8.0.
<https://docs.percona.com/percona-xtrabackup/8.0/docker.html>

4. Redes Docker: *bridge*, *host* y redes personalizadas

El subsistema de red de Docker, *libnetwork*, ofrece varios *drivers* para conciliar rendimiento, seguridad y topología. Por defecto, el driver **bridge** crea la interfaz `docker0`. Cada contenedor recibe una IP privada (*RFC 1918*) y su nombre se resuelve vía un DNS *lightweight* embebido en el demonio. Las reglas DNAT/SNAT generadas en iptables habilitan exposición de puertos (`-p 8080:80`) y salida a Internet.

Cuando la aplicación exige latencia mínima o `ports < 1024`, podemos optar por **host**. Este *driver* elimina NAT y mapea los `sockets` del contenedor directamente sobre los del sistema operativo anfitrión. El precio: pierde aislamiento; un proceso comprometido puede leer tráfico de otros servicios. Mitigación: perfiles AppArmor/SELinux estrictos y caída de capacidades de red avanzadas.

Para entornos *multi-host*, Docker (en modo Swarm) y Kubernetes emplean el driver **overlay**, que encapsula paquetes en VXLAN. Cada nodo mantiene una tabla endpoint-IP; el tráfico se cifra opcionalmente (IPSec) y se direcciona mediante un puerto UDP (4789). Se sugiere ajustar MTU a 1450 para evitar fragmentación en redes Ethernet estándar.

macvlan da al contenedor una MAC e IP visibles en la LAN física. Facilita integrar *appliances legacy* que requieren *broadcast* o DHCP. La complejidad aumentará en redes con filtros de puertos-seguridad o cuando el *switch* limite MACs por puerto.

La **cadena DOCKER-USER** permite al administrador injertar reglas globales antes de que Docker inserte DNAT/SNAT. Allí se pueden bloquear rangos, aplicar *rate-limit* o registrar métricas para auditoría. Otra opción es `--icc=false`, que impide tráfico automático entre contenedores del mismo bridge, obligando a pasar por *proxies* explícitos.

Rendimiento: pruebas de laboratorio muestran que NAT de bridge añade 5–7 % de CPU y ~1 ms de latencia, mientras que *macvlan* y *host* se acercan al *wire-speed*. Sin embargo, el 80 % de los microservicios HTTP nunca notará esa penalización; la optimización se vuelve importante en *streaming*, HFT y bases de datos OLTP.

Recursos para profundizar:

- **Docker** © 2013-2025. Docker Docs “Networking overview”.
<https://docs.docker.com/network/>
- White paper “VXLAN performance tuning in overlay networks”, Broadcom 2022.

5. Docker Compose y registries privados

Docker Compose simplifica el lanzamiento de pilas multi-contenedor mediante un archivo YAML. Secciones **services**, **volumes** y **networks** describen la topología; **depends_on** ordena arranque, mientras que **healthcheck** permite validar disponibilidad. Con Compose v2, la CLI se integra a Docker y soporta **docker compose up, ps, logs, down**.

En desarrollo, Compose permite uniformar entornos: “clona, **up -d**, listo”. En integración continua, se usa para pruebas *end-to-end*: un *job* levanta la pila, corre *test* y baja todo. En producción, Compose se sustituye por Swarm o Kubernetes, que ofrecen escalado, HA y actualizaciones continuas. Sin embargo, Compose sigue sirviendo como DSL de alto nivel que más tarde se traduce a manifiestos Helm o Kustomize.

La otra pieza clave es el **registry privado**. Subir imágenes a Docker Hub público puede violar compliance o exponer secretos. Herramientas como Harbor (open source), GitLab Registry o ECR ofrecen autenticación, políticas de retención, replicación geográfica y, sobre todo, **escaneo** y **firma**. El escaneo automático detecta CVEs; la firma (Notary v2 o cosign) certifica que la imagen proviene de un pipeline confiable. Gatekeeper o Kyverno pueden rechazar deployments de imágenes sin firma válida.

Política típica:

- Escanear cada *push*: bloquear CVE crítico sin parche.
- Exigir firma cosign antes de *merge* a rama **main**.
- Retención: últimos 30 *tags* de cada microservicio. Garbage Collection semanal.
- *Pull-through cache*: si Docker Hub limita tus *pulls*, *mirror* local para capas base (Alpine, *distroless*).

Costos y soberanía: ECR cobra por GB-mes; Harbor requiere infraestructura propia pero garantiza jurisdicción. La elección dependerá de las normativas (GDPR, Ley Arg. 25 326), presupuesto y SLA interno.

Para la cadena de suministro, la firma complementa TLS: mientras que TLS solo asegura transporte, *cosign* verifica que nadie modificó la imagen —incluso dentro del *registry*— después de ser firmada. Durante auditorías de seguridad, presentar la firma más el *digest* inmutable cierra el eslabón entre commit Git y contenedor en producción.

Recursos para profundizar:

- **Compose Spec oficial:** <https://compose-spec.io/>
- **Harbor © (2025).** Harbor docs – firma y escaneo: <https://goharbor.io/docs/>
- **Sigstore cosign tutorial (CLI & GitHub Actions):** <https://docs.sigstore.dev/cosign>
- **CNCF** [Cloud Native Computing Foundation]. (2021, may 14). Notary v2: Supply Chain Security for Containers - Justin Cormack, Docker & Steve Lasker, Microsoft. <https://www.youtube.com/watch?v=SZMbuiREQVU>

Bibliografía

- Aqua Security. “Container Image Scanning Best Practices”, 2024.
- Compose Specification – <https://compose-spec.io/>
- Documentación Docker – <https://docs.docker.com/>
- Merkel, D. “Docker: Lightweight Linux Containers...”, *Linux Journal*, 2014.
- Poulton, N. *Docker Deep Dive*, 5.^a ed., 2023.
- Sigstore. *cosign User Guide* – <https://docs.sigstore.dev/cosign/overview/>
- Turnbull, J. *The Docker Book*, 2022.